

POO - PARTIE I - Fondations de la programmation orientee objet

Resume complet des sections 2.1 a 2.5 (niveau debutant a expert)

Ce document synthétise les cinq tutoriels de la section 2. **Les 4 piliers (vision moderne) : 2.1 Encapsulation, 2.2 Abstraction, 2.3 Heritage, 2.4 Polymorphisme et 2.5 Piliers vs pratiques actuelles.** L'objectif est de conserver l'essentiel des principes, des choix de conception et des heuristiques d'architecture utilisables dans des projets reels.

Fil conducteur moderne

*La POO moderne n'oppose pas theorie et pratique. Elle met l'accent sur : **protéger les invariants, exposer des contrats stables, composer plutot qu'heritier par reflexe, et isoler les variations avec le polymorphisme.***

2.1 Encapsulation - Protéger les invariants

- **Definition utile** : encapsuler, ce n'est pas juste cacher des champs; c'est empêcher qu'un objet entre dans un etat invalide.
- **Invariant** : regle toujours vraie (ex: solde ≥ 0 , email valide, quantite > 0).
- **Ou appliquer les controles** : constructeur/factory, methodes de mutation, deserialisation, frontieres applicatives.
- **Vision moderne** : preferer des methodes metier explicites (deposer, retirer, confirmer) plutot que des setters generiques.
- **Danger classique** : fuite de mutabilite (retourner une liste interne modifiable).
- **Techniques cle** : read-only, copies defensives, value objects, validation centralisee, tests de violations d'invariants.

Conclusion 2.1 : un objet bien encapsule est auto-protecteur et plus fiable sous evolution.

2.2 Abstraction - Exposer le necessaire, masquer l'accidentel

- **But** : separer le *quoi* (contrat) du *comment* (implementation).
- **Necessaire** : operations metier, garanties, erreurs explicites.
- **Accidentel** : DB, SDK, protocoles, optimisations internes, details temporaires.
- **Outils** : interfaces, classes abstraites ciblees, modules, adapters, facades.
- **Criteres d'une bonne abstraction** : minimale mais complete, nommage metier, stabilite temporelle, semantics claires.
- **Fuites d'abstraction** : exposer SQLException, codes HTTP bruts ou concepts infra dans le domaine.

- **Pragmatique** : abstraire quand la variabilité est réelle (testabilité, changement techno, frontière architecture).

Conclusion 2.2 : une bonne abstraction diminue le couplage et augmente la durée de vie du code.

2.3 Heritage - Reutilisation vs hierarchie de types

- **Idee centrale** : l'heritage sert d'abord au sous-typage valide, pas à la reutilisation rapide.
- **Test décisif** : substituabilité (LSP) - un enfant doit remplacer le parent sans surprise.
- **Quand c'est pertinent** : relation est-une claire, contrat stable, points d'extension contrôlés.
- **Quand éviter** : heritage pour utilitaires, base class géante, hiérarchie profonde, overrides imprévisibles.
- **Alternative privilégiée** : composition (assembler des comportements via composants injectés).
- **Bonnes pratiques** : classe parent petite, méthodes finales pour invariants, sealed types si possible, tests de contrat.

Conclusion 2.3 : composition d'abord; heritage quand le modèle de types l'exige vraiment.

2.4 Polymorphisme - Substituabilite, interfaces, dispatch

- **Definition** : un meme contrat, plusieurs implementations interchangeables.
- **Triple axe** : substituabilite (LSP), interfaces (contrat), dispatch (choix de l'implementation executee).
- **Valeur architecture** : isoler les variations (paiement, stockage, notification, pricing) sans if/switch geants.
- **Dispatch** : dynamique en POO classique (la methode concrete est choisie a l'execution).
- **Integration DI** : le code depend d'interfaces; l'environnement injecte la bonne implementation.
- **Qualite** : erreurs harmonisees au niveau du contrat, pas de fuite SDK/infra.
- **Tests** : contract tests communs + tests techniques propres a chaque implementation.

Conclusion 2.4 : le polymorphisme bien cadre rend le systeme extensible sans casser le client.

2.5 Les 4 piliers vs pratiques actuelles : composition, immutabilite, FP

- **Pas d'opposition stricte POO vs FP** : les deux paradigmes se combinent efficacement.
- **Composition** : norme actuelle pour assembler des comportements avec un couplage faible.
- **Immutabilite** : reduit les etats invalides transitoires et simplifie concurrence/debug.
- **FP dans OO** : fonctions pures pour calculs, Result/Either pour erreurs, pipelines de transformations.
- **Relecture des piliers** : encapsulation = invariants; abstraction = frontieres stables; heritage = usage cible; polymorphisme = variation maitrisee.
- **Architecture hybride recommandee** : domaine explicite + ports/adapters + calculs purs + types metier forts.

Conclusion 2.5 : la POO moderne est pragmatique, orientee lisibilite, testabilite et evolutivite.

Grille de decision rapide (application immediate)

- 1 Si le risque principal est la **coherence metier** -> renforcer encapsulation + invariants.
- 2 Si la technologie peut changer -> introduire abstraction (port + adapter).
- 3 Si tu veux juste reutiliser du code -> preferer composition.
- 4 Si plusieurs comportements doivent coexister proprement -> polymorphisme par interface/strategie.
- 5 Si un calcul est deterministe et sans effet de bord -> fonction pure.
- 6 Si l'etat est partage/concurrent -> immutabilite ou synchronisation stricte.

Synthese finale

Les 4 piliers restent fondamentaux, mais leur application moderne privilegie la clarté des contrats, la protection des invariants, la composition des comportements et la maitrise des variations. Un design reussi n'est ni dogmatiquement OO ni dogmatiquement FP : il choisit les bons outils au bon niveau.

Document genere automatiquement - Resume pedagogique consolide.